

LAPORAN PARKTIKUM 2

SISTEM OPERASI



Dosen Pengampu Mata Kuliah:

Triawan Adi Cahyanto, M.Kom

Disusun Oleh:

Rafli Musthofa (2410651019)

PRODI TEKNIK INFORMATIKA

FAKULTAS TEKNIK

UNIVERSITAS MUHAMMADIYAH JEMBER

2025

TUJUAN

1. Memahami konsep proses dan siklus hidupnya.
2. Mampu membuat dan mengelola proses menggunakan system call seperti `fork()`, `wait()`, dan `exec()`.
3. Memahami hubungan antara parent process dan child process.
4. Menganalisis Process ID (PID) dan Process Control Block (PCB).
5. Mempelajari komunikasi antar proses (Interprocess Communication / IPC) menggunakan *pipe* dan *shared memory*.
6. Membandingkan performa metode IPC

Dasar Teori

A. Konsep Proses

Proses adalah program yang sedang dieksekusi oleh sistem operasi. Setiap proses memiliki identitas unik yang disebut Process ID (PID). Dalam sistem operasi seperti Linux, proses dapat membuat proses lain menggunakan system call `fork()`. Proses yang membuat disebut *parent*, sedangkan proses hasil duplikasi disebut *child process*.

B. Siklus Hidup Proses

Proses mengalami beberapa status selama hidupnya, yaitu *new*, *ready*, *running*, *waiting*, dan *terminated*. Sistem operasi mengatur transisi antar status melalui scheduler.

C. System Call `fork()` dan `wait()`

- `fork()` digunakan untuk membuat proses baru (child).
- `wait()` digunakan untuk membuat proses baru (child).

D. Interprocess Communication (IPC)

IPC adalah mekanisme komunikasi antar proses. Metode yang umum digunakan antara lain:

- Pipes – komunikasi satu arah antar proses parent-child.
- Named Pipes (FIFO) – versi pipe dengan nama yang bisa diakses banyak proses.
- Shared Memory – proses berbagi area memori yang sama untuk bertukar data, umumnya lebih cepat dari pipe.

PERCOBAAN

1. Percobaan 1.1

Membuat proses sederhana kode program (process_basic.c):

```
#include <stdio.h>

#include <unistd.h>

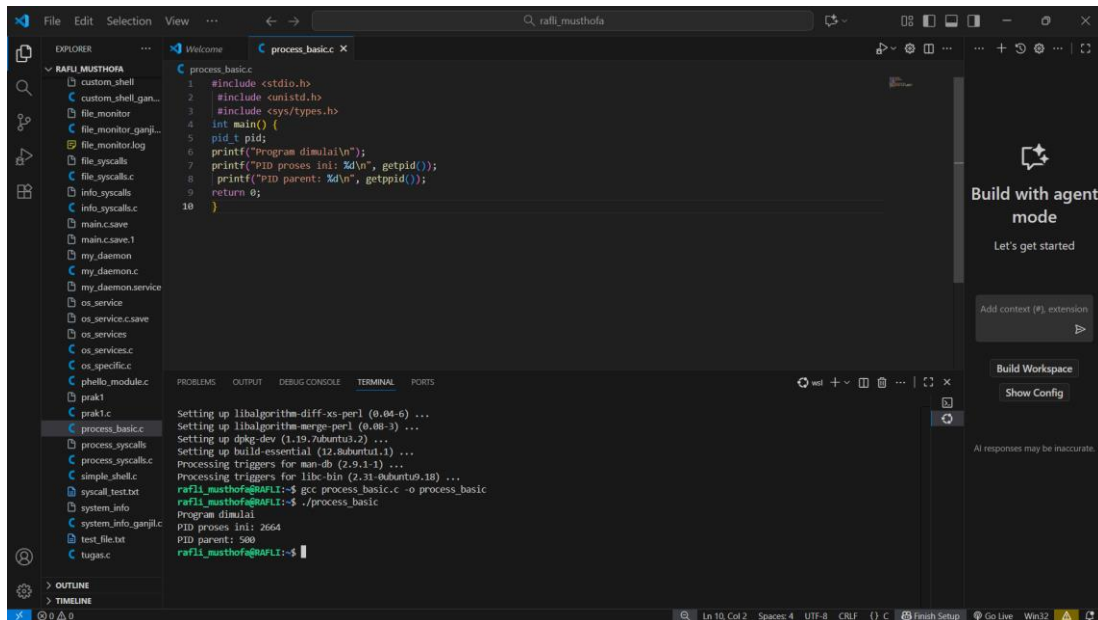
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Program dimulai\n");
    printf("PID proses ini: %d\n", getpid());
    printf("PID parent: %d\n", getppid());
    return 0;
}
```

Analisis:

Program mencetak PID proses saat ini dan PID parent-nya. Ini menunjukkan bahwa setiap proses di Linux memiliki identitas unik, dan parent-nya adalah proses shell yang menjalankannya.



The screenshot shows a code editor with the file `process_basic.c` open. The code is as follows:

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 int main() {
5     pid_t pid;
6     printf("Program dimulai\n");
7     printf("PID proses ini: %d\n", getpid());
8     printf("PID parent: %d\n", getppid());
9     return 0;
10 }
```

The terminal output shows the following commands and results:

```
Setting up libalgorithm-diff-xs-perl (0.04-6) ...
Setting up libalgorithm-merge-perl (0.08-3) ...
Setting up dpkg-dev (1.19.7ubuntu3.2) ...
Setting up build-essential (12.7ubuntu1) ...
Processing triggers for man-db (2.9.1-1) ...
Processing triggers for libc-bin (2.31-0ubuntu1.18) ...
rafl1_musthof@RAFL1:~$ gcc process_basic.c -o process_basic
rafl1_musthof@RAFL1:~$ ./process_basic
Program dimulai
PID proses ini: 2664
PID parent: 500
rafl1_musthof@RAFL1:~$
```

2. Percobaan 1.2

Parent dan Child Process (fork_demo.c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>

int main() {
    pid_t pid;

    printf("Sebelum fork()\n");
    printf("PID: %d\n\n", getpid());

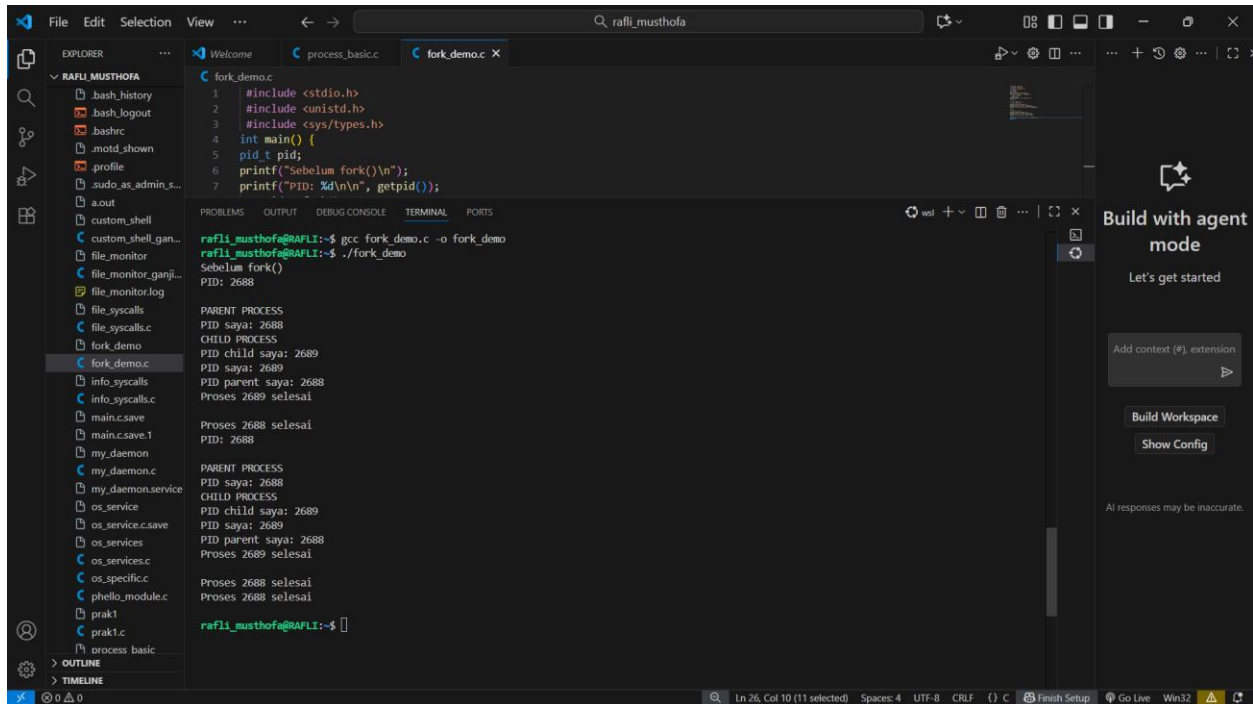
    pid = fork();

    if (pid < 0) {
        fprintf(stderr, "Fork gagal!\n");
        return 1;
    } else if (pid == 0) {
        printf("CHILD PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID parent saya: %d\n", getppid());
    } else {
        printf("PARENT PROCESS\n");
        printf("PID saya: %d\n", getpid());
        printf("PID child saya: %d\n", pid);
    }

    printf("Proses %d selesai\n\n", getpid());
    return 0;
}
```

Analisis:

Setelah fork() dijalankan, akan ada dua proses: parent dan child. Masing-masing mengeksekusi bagian kode yang berbeda tergantung nilai pid. Perbedaan PID menunjukkan proses yang terpisah.



The screenshot shows a VS Code editor with a file explorer on the left, a code editor in the center, and a terminal at the bottom. The code editor displays a C program named `fork_demo.c` that uses `fork()` to create a child process. The terminal shows the output of the program, including the parent and child PIDs and the status of the processes.

```
1 #include <stdio.h>
2 #include <unistd.h>
3 #include <sys/types.h>
4 int main() {
5     pid_t pid;
6     printf("Sebelum fork()\n");
7     printf("PID: %d\n", getpid());
8     pid = fork();
9     if (pid == 0) {
10         printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
11         sleep(3);
12     } else {
13         printf("Parent: PID saya %d\n", getpid());
14     }
15 }
```

Terminal Output:

```
rafl_i_musthofa@RAFLI:~$ gcc fork_demo.c -o fork_demo
rafl_i_musthofa@RAFLI:~$ ./fork_demo
Sebelum fork()
PID: 2688

PARENT PROCESS
PID saya: 2688
CHILD PROCESS
PID child saya: 2689
PID saya: 2689
PID parent saya: 2688
Proses 2689 selesai

Proses 2688 selesai
PID: 2688

PARENT PROCESS
PID saya: 2688
CHILD PROCESS
PID child saya: 2689
PID saya: 2689
PID parent saya: 2688
Proses 2689 selesai

Proses 2688 selesai
Proses 2688 selesai

rafl_i_musthofa@RAFLI:~$
```

3. Percobaan 1.3

Process termination dengan wait() kode program (process_wait.c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/types.h>
#include <sys/wait.h>

int main() {
    pid_t pid;
    int status;

    pid = fork();

    if (pid == 0) {
        printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
        sleep(3);
    } else {
        wait(&status);
        printf("Parent: PID saya %d, status anak %d\n", getpid(), status);
    }
}
```

```

        sleep(3);
        printf("Child: Selesai!\n");
        return 42;
    } else {
        printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
        wait(&status);

        if (WIFEXITED(status)) {
            printf("Parent: Child selesai dengan exit code: %d\n",
                WEXITSTATUS(status));
        }
    }
    return 0;
}

```

Analisis:

Parent menunggu proses anak selesai menggunakan wait(). Ketika anak selesai, parent menerima status keluar (exit code). Ini menggambarkan sinkronisasi antara parent dan child.

The screenshot shows a Visual Studio Code editor with a project named 'RAFIL MUSTHOFIA'. The file explorer on the left lists various files, including 'process_wait.c'. The main editor window displays the source code of 'process_wait.c', which is a C program demonstrating process synchronization using the 'wait()' function. The code includes headers for `stdio.h`, `unistd.h`, `sys/types.h`, and `sys/wait.h`. The `main()` function forks a child process, which sleeps for 3 seconds and then prints 'Child: Selesai!\n' before returning an exit code of 42. The parent process then calls `wait(&status)` to wait for the child to finish. Once the child finishes, the parent checks if the process has exited (`WIFEXITED(status)`) and prints the exit code (`WEXITSTATUS(status)`). The terminal at the bottom shows the compilation and execution of the program, with the output matching the expected behavior: 'Parent: Menunggu child (PID: 2707) selesai...', 'Child: PID saya 2707, akan tidur 3 detik', 'Child: Selesai!', and 'Parent: child selesai dengan exit code: 42'.

```

1  #include <stdio.h>
2  #include <unistd.h>
3  #include <sys/types.h>
4  #include <sys/wait.h>
5  int main() {
6      pid_t pid;
7      int status;
8      pid = fork();
9      if (pid == 0) {
10         // Child process
11         printf("Child: PID saya %d, akan tidur 3 detik\n", getpid());
12         sleep(3);
13         printf("Child: Selesai!\n");
14         return 42; // Exit code
15     }
16     else {
17         // Parent process
18         printf("Parent: Menunggu child (PID: %d) selesai...\n", pid);
19         wait(&status);
20         if (WIFEXITED(status)) {
21             printf("Parent: child selesai dengan exit code: %d\n",
22                 WEXITSTATUS(status));
23         }
24     }
25     return 0;
26 }

```

```

compilation terminated.
rafli_musthofa@AFIL:~$ gcc process_wait.c -o process_wait
rafli_musthofa@AFIL:~$ ./process_wait
Parent: Menunggu child (PID: 2707) selesai...
Child: PID saya 2707, akan tidur 3 detik
Child: Selesai!
Parent: child selesai dengan exit code: 42
rafli_musthofa@AFIL:~$

```

4. Percobaan 1.4

Multiple child processes kode program (multiple_fork.c)

```
#include <stdio.h>
#include <unistd.h>
#include <sys/wait.h>

int main() {
    int i;
    pid_t pid;
    int num_children = 3;

    printf("Parent PID: %d\n\n", getpid());

    // Membuat beberapa child process
    for (i = 0; i < num_children; i++) {
        pid = fork();

        if (pid == 0) {
            // Proses anak
            printf("Child %d: PID = %d, Parent PID = %d\n",
                i + 1, getpid(), getppid());
            sleep(i + 1); // simulasi kerja
            printf("Child %d selesai\n", i + 1);
            return 0; // anak selesai, keluar dari loop dan program anak
        }
    }

    // Proses parent menunggu semua child selesai
    for (i = 0; i < num_children; i++) {
        int status;
        pid_t child_pid = wait(&status);
```

```

printf("Parent: Child dengan PID %d selesai\n", child_pid);
}

printf("\nParent: Semua child selesai\n");
return 0;
}

```

Analisis:

Program membuat tiga child process yang berjalan secara paralel. Parent menunggu masing-masing child menggunakan `wait()`. Urutan selesai bergantung pada durasi `sleep()`.

```

multiple_fork.c
5  int main() {
13     for (i = 0; i < num_children; i++) {
25
26     // Proses parent menunggu semua child selesai
27     for (i = 0; i < num_children; i++) {
28         int status;
29         pid_t child_pid = wait(&status);

```

```

raflimusthofa@RAFLI:~$ ./multiple_fork
Parent PID: 2735
Child 1: PID = 2736, Parent PID = 2735
Child 2: PID = 2737, Parent PID = 2735
Child 3: PID = 2738, Parent PID = 2735
Child 1 selesai
Parent: Child dengan PID 2736 selesai
Child 2 selesai
Parent PID: 2735
Child 1: PID = 2736, Parent PID = 2735
Child 2: PID = 2737, Parent PID = 2735
Child 3: PID = 2738, Parent PID = 2735
Child 1 selesai
Parent: Child dengan PID 2736 selesai
Child 2 selesai
Parent: Child dengan PID 2736 selesai
Child 2 selesai
Child 2 selesai
Parent: Child dengan PID 2737 selesai
Child 3 selesai
Parent: Child dengan PID 2738 selesai
Parent: Semua child selesai
raflimusthofa@RAFLI:~$

```


1. Percobaan 2.1

Pipes (One Way Communication) kode program (pipe_demo.c)

```
#include <stdio.h>
#include <unistd.h>
#include <string.h>

int main() {
    int pipefd[2];
    pid_t pid;
    char write_msg[] = "Hello dari Parent!";
    char read_msg[100];
    pipe(pipefd);
    pid = fork();
    if (pid == 0) {
        close(pipefd[1]);
        read(pipefd[0], read_msg, sizeof(read_msg));
        printf("Child menerima: %s\n", read_msg);
        close(pipefd[0]);
    } else {
        close(pipefd[0]);
        printf("Parent mengirim: %s\n", write_msg);
        write(pipefd[1], write_msg, strlen(write_msg) + 1);
        close(pipefd[1]);
        wait(NULL);
    }
    return 0;
}
```

Analisis:

Parent menulis pesan ke pipe, dan child membacanya. Komunikasi ini bersifat satu arah dan hanya terjadi antar parent–child.

```
1 #include <stdio.h>
2 #include <string.h>
3 #include <sys/wait.h>
4
5 int main() {
6     int pipefd[2]; // pipefd[0] = read, pipefd[1] = write
7     pid_t pid;
8     char write_msg[] = "Hello dari Parent!";
9     char read_msg[100];
10    // Buat pipe
11    if (pipe(pipefd) == -1) {
12        perror("Pipe gagal");
13        return 1;
14    }
15    pid = fork();
16    if (pid == 0) {
17        // child process - membaca dari pipe
18        close(pipefd[1]); // Tutup write end
19        read(pipefd[0], read_msg, sizeof(read_msg));
20        printf("Child menerima: %s\n", read_msg);
21        close(pipefd[0]);
22    }
23    else {
24        // Parent process - menulis ke pipe
25        close(pipefd[0]); // Tutup read end
26        write(pipefd[1], write_msg, sizeof(write_msg));
27        close(pipefd[1]);
28    }
29    wait(NULL);
30    return 0;
31}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
rafli_musthofa@RAFLI:~$ ./pipe_demo
Parent mengirim: Hello dari Parent!
Child menerima: Hello dari Parent!
rafli_musthofa@RAFLI:~$
```

2. Percobaan 2.2

Named Pipes (FIFO) program: `fifo_writer.c` dan `fifo_reader.c`

Kedua program berkomunikasi melalui FIFO file `/tmp/my_fifo`.

```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <fcntl.h>
4 #include <sys/stat.h>
5 #include <unistd.h>
6 #define FIFO_FILE "/tmp/my_fifo"
7
8 int main() {
9     int fd;
10    char buffer[100];
11    printf("Reader: Membuka FIFO...\n");
12    fd = open(FIFO_FILE, O_RDONLY);
13    read(fd, buffer, sizeof(buffer));
14    printf("Reader: Menerima pesan: %s\n", buffer);
15    close(fd);
16    unlink(FIFO_FILE); // Hapus FIFO
17    return 0;
18}
```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

```
-bash: ./fifo_reader: No such file or directory
rafli_musthofa@RAFLI:~$ ./fifo_reader
Reader: Membuka FIFO...
Reader: Menerima pesan:
rafli_musthofa@RAFLI:~$
```

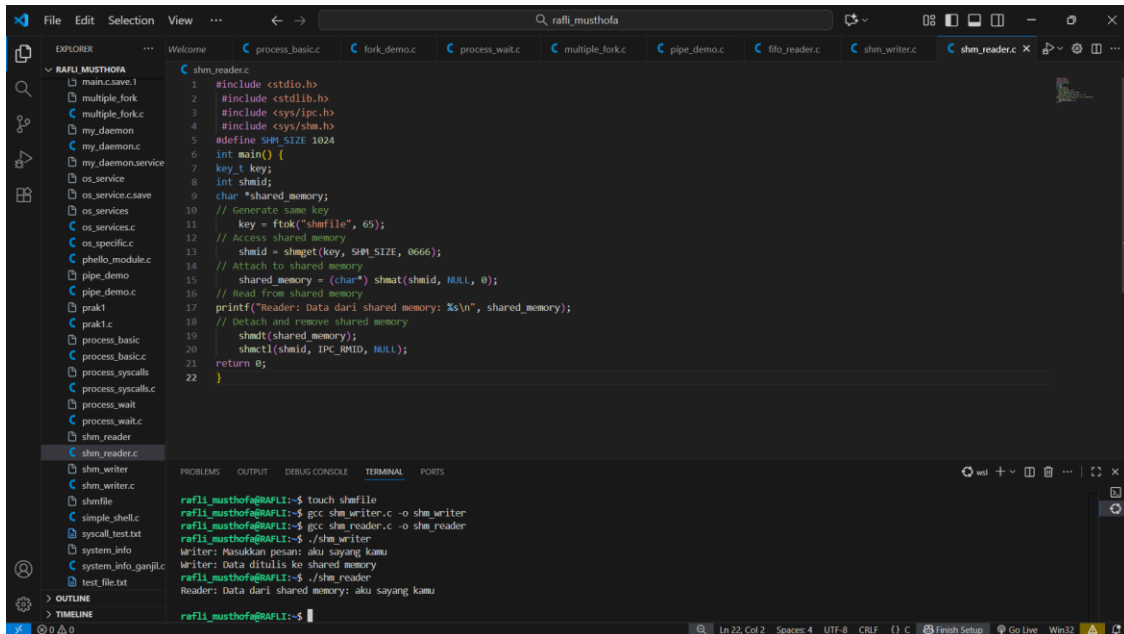
Analisis:

Named Pipe memungkinkan dua proses berbeda (tidak harus parent-child) untuk berkomunikasi. FIFO bersifat *persistent* di filesystem hingga dihapus dengan `unlink()`.

3. Percobaan 2.3

Shared Memory IPC kode program (shm_writer.c dan shm_reader.c)

Writer menulis pesan ke shared memory, dan reader membacanya.



```
1 #include <stdio.h>
2 #include <stdlib.h>
3 #include <sys/ipc.h>
4 #include <sys/shm.h>
5 #define SHM_SIZE 1024
6 int main() {
7     key_t key;
8     int shmid;
9     char *shared_memory;
10    // Generate new key
11    key = ftok("shmfile", 65);
12    // Access shared memory
13    shmid = shmget(key, SHM_SIZE, 0666);
14    // Attach to shared memory
15    shared_memory = (char*) shmat(shmid, NULL, 0);
16    // Read from shared memory
17    printf("Reader: Data dari shared memory: %s\n", shared_memory);
18    // Detach and remove shared memory
19    shmdt(shared_memory);
20    shmctl(shmid, IPC_RMID, NULL);
21    return 0;
22 }
```

```
refli_musthofa@RAFI:~$ touch shmfile
refli_musthofa@RAFI:~$ gcc shm_writer.c -o shm_writer
refli_musthofa@RAFI:~$ gcc shm_reader.c -o shm_reader
refli_musthofa@RAFI:~$ ./shm_writer
Writer: Masukkan pesan: aku sayang kamu
refli_musthofa@RAFI:~$ ./shm_reader
Reader: Data dari shared memory: aku sayang kamu
```

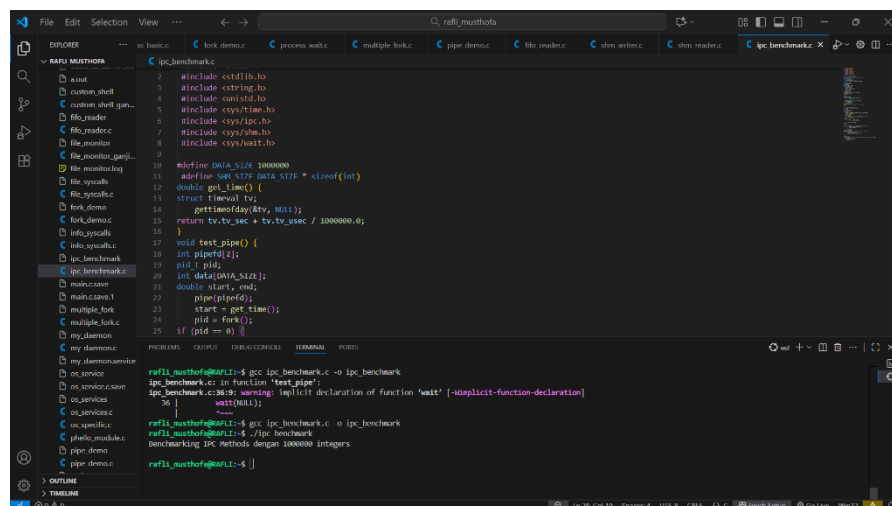
Analisis:

Shared Memory memungkinkan pertukaran data lebih cepat dibanding pipe karena data langsung dibaca dari ruang memori yang sama, tanpa perlu salinan antar proses.

4. Percobaan 2.4

Perbandingan performa IPC kode (ipc_benchmark.c)

Program ini mengukur waktu transef data menggunakan pipe dan shared memory



```
1 #include <stdio.h>
2 #include <string.h>
3 #include <unistd.h>
4 #include <sys/time.h>
5 #include <sys/ipc.h>
6 #include <sys/shm.h>
7 #include <sys/wait.h>
8 #include <sys/types.h>
9
10 #define DATA_SIZE 1000000
11 #define SHM_SIZE DATA_SIZE * sizeof(int)
12 double get_time() {
13     struct timeval tv;
14     gettimeofday(&tv, NULL);
15     return tv.tv_sec + tv.tv_usec / 1000000.0;
16 }
17 void test_pipe() {
18     int pipefd[2];
19     pid_t pid;
20     int data[DATA_SIZE];
21     double start, end;
22     pipe(pipefd);
23     start = get_time();
24     pid = fork();
25     if (pid == 0) {
26         // ...
27     }
28 }
```

```
refli_musthofa@RAFI:~$ gcc ipc_benchmark.c -o ipc_benchmark
ipc_benchmark.c: In function 'test_pipe':
ipc_benchmark.c:36: warning: implicit declaration of function 'wait' [-Wimplicit-function-declaration]
36 |     wait(NULL);
   |     ^~~~~
refli_musthofa@RAFI:~$ gcc ipc_benchmark.c -o ipc_benchmark
refli_musthofa@RAFI:~$ ./ipc_benchmark
Benchmarking IPC Methods dengan 1000000 integers
refli_musthofa@RAFI:~$
```

Analisis:

Hasil menunjukkan bahwa komunikasi menggunakan shared memory secara signifikan lebih cepat dibanding pipe, terutama untuk ukuran data besar, karena tidak ada overhead sistem kernel dalam pengiriman data.

KESIMPULAN

1. Proses adalah satuan kerja dasar dalam sistem operasi yang dapat membuat proses lain menggunakan fork().
2. Hubungan parent-child dapat diamati melalui PID dan wait() yang digunakan untuk sinkronisasi.
3. Interprocess Communication (IPC) penting untuk pertukaran data antar proses.
4. *Pipes* cocok untuk komunikasi sederhana, sementara *shared memory* lebih efisien untuk volume data besar.
5. Melalui percobaan ini, mahasiswa memahami mekanisme dasar pengelolaan proses dan komunikasi antar proses di sistem operasi Linux.